NASM32 Assembly Programming Laboratory

Four programming problems | Conditional jumps, procedures, validation, and arithmetic

Student:Francis Karl G. Yulo Section: TN25 Date: _____

General Instructions

• Develop four separate 32-bit NASM assembly programs. Use the instructor-provided template and the required operating-system conventions.

• Use named procedures (CALL/RET) and conditional/unconditional jumps to implement control flow. Use .data, .bss, and .text sections as appropriate.

• Read full names with spaces. Strip input line endings before displaying text. For numeric inputs, perform ASCII-to-integer conversion and validate malformed input.

• For each problem, submit one .asm source file and screenshots showing the compilation/linking commands and all required test-case outputs.

• Do not submit screenshots alone. Include brief written answers to the analysis questions for each problem.

Compilation example (32-bit Linux, NASM + ld): nasm -f elf32 problem1.asm -o problem1.o | ld -m elf_i386 problem1.o -o problem1 | ./problem1

Note: On other operating systems or when using C-library functions, compilation/linking commands may differ. Use the method demonstrated in class.

Problem 1 — Name, Age, and Gender Greeting

Difficulty: Basic | Objective: Practice input, character comparisons, jumps, and procedures.

Write a 32-bit NASM program that asks for the user's full name, age, and gender (M or F). Print "Hello <name>" and "You are <age> years old". Then print "Hello" for M, "World" for F, and "INVALID" for any other input. Accept lowercase m/f as equivalent to uppercase (extension).

Required test cases

```
What is your name? Alfred Hitchcock
How old are you? 12
What is your gender? M
Hello Alfred Hitchcock
You are 12 years old
Hello
```

Repeat with F (final line: World) and X (final line: INVALID).

Student questions

• Which conditional jump distinguishes M from F?

At mov al,[Gender] conditional jump will go through each condition if it doesn't match the case letter if it lower case or upper case "m" but if it is either it will then check "F" it is the same procedure but it will store the gender in the array

• What happens when the gender is neither M nor F?

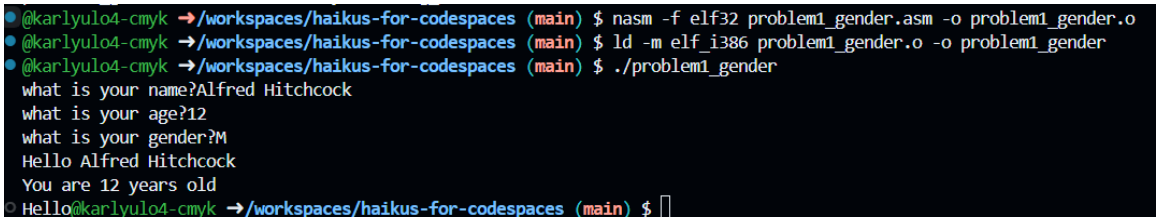
It will print "INVALID" based on the conditional jump if it Is neither then it will jump to the print_invalid

• Why should the name be stored in a buffer rather than a single byte?

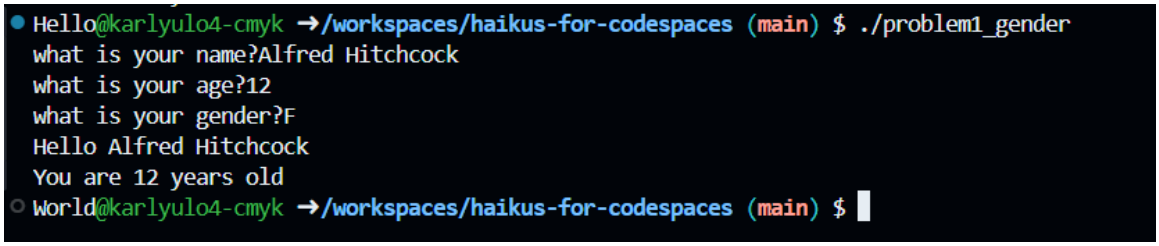
It will make things easier to store the name of the person and it will remove any possibility of error by making sure that it is being read while a byte is only 1 character that is why we need to use a Buffer

PROOF OF EXECUTION

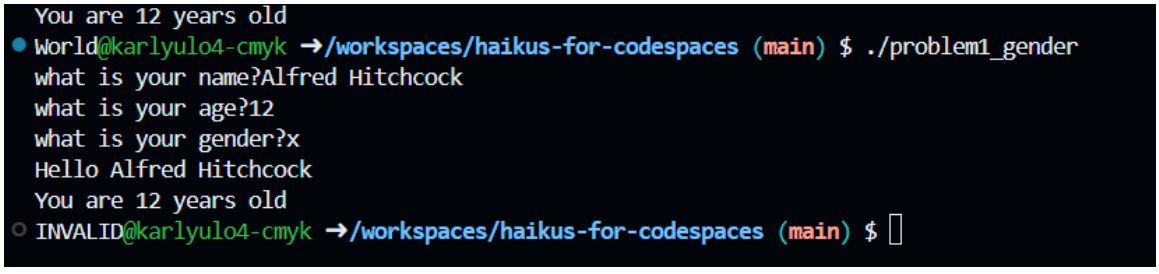
Test case 1:



Test case 2: with f



Test case 3:with X



Problem 2 — Student Year-Level Classification

Difficulty: Basic–Intermediate | Objective: Implement a multi-branch selection.

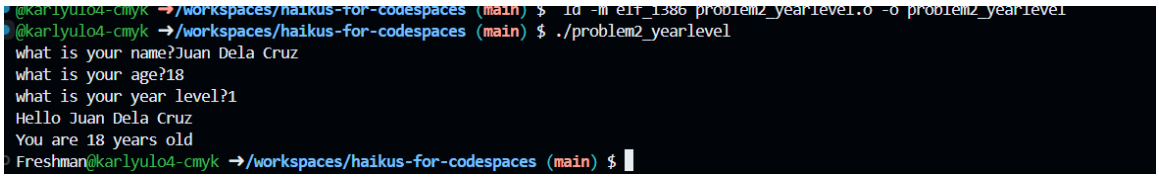
Ask for full name, age, and year level. Display "Hello <name>" and "You are <age> years old". Classify 1 as Freshman, 2 as Sophomore, 3 as Junior, 4 as Senior; otherwise display "INVALID YEAR LEVEL".

Required test cases

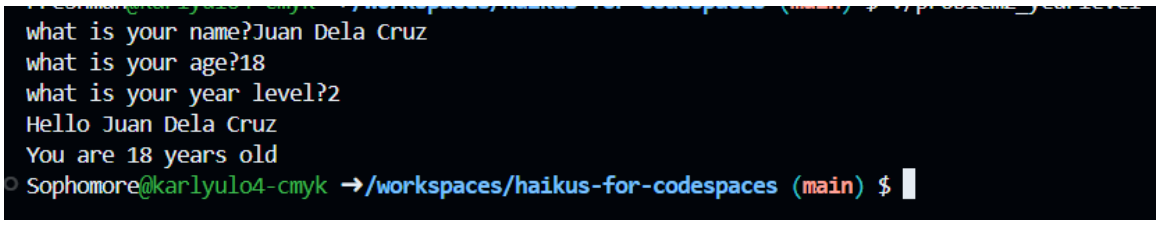
```
What is your name? Juan Dela Cruz
How old are you? 18
What is your year level? 1
Hello Juan Dela Cruz
You are 18 years old
Freshman
```

Also test levels 2, 3, 4, and 5; expected final lines: Sophomore, Junior, Senior, and INVALID YEAR LEVEL, respectively.

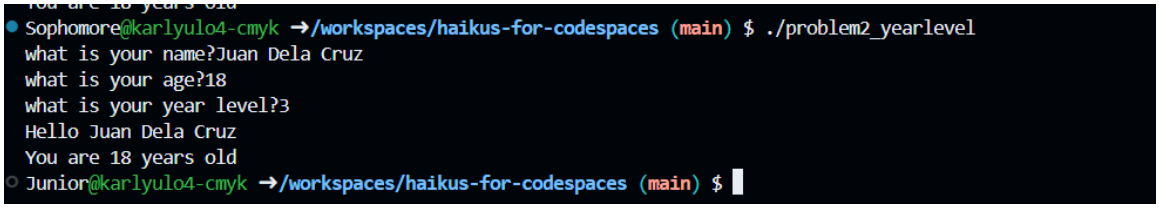
Test 1



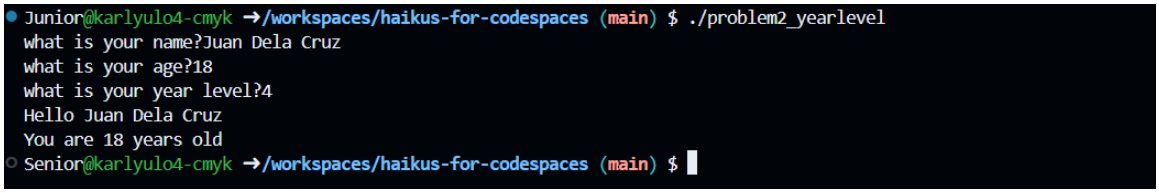
Test 2



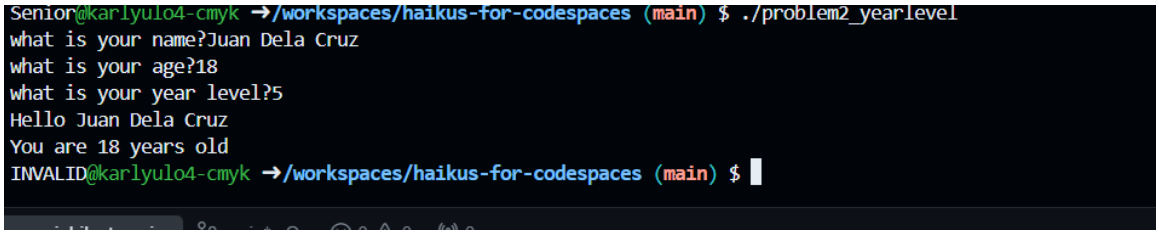
Test 3



Test 4



Test 5



Student questions

• How many comparisons are needed in your chosen control-flow design?

4 comparisons because at the end its like and if else statement where if all of them fail it goes to the last one which will print invalid

• How do you prevent execution from falling through into another classification?

i do that by creating the conditional jump in order for them to be categorized and if that is met it will do ".do_print" in order for it to be printed or outputting the message or classification of the year level

• How does your program handle an invalid year-level input?

It handles it by if the conditional jumps doesn't match like 1,2,3, and 4 then it will jump to invalid

So that it does the same ".do_print" but it will print "INVALID" in the last line of the output

Problem 3 — Student Grade Evaluation

Difficulty: Intermediate | Objective: Compare numeric ranges and validate input.

Ask for full name, student ID, and integer grade. Display the entered details and the correct classification. Grades outside 0–100 or nonnumeric grades must be rejected. Treat the student ID as text so leading zeros are preserved.

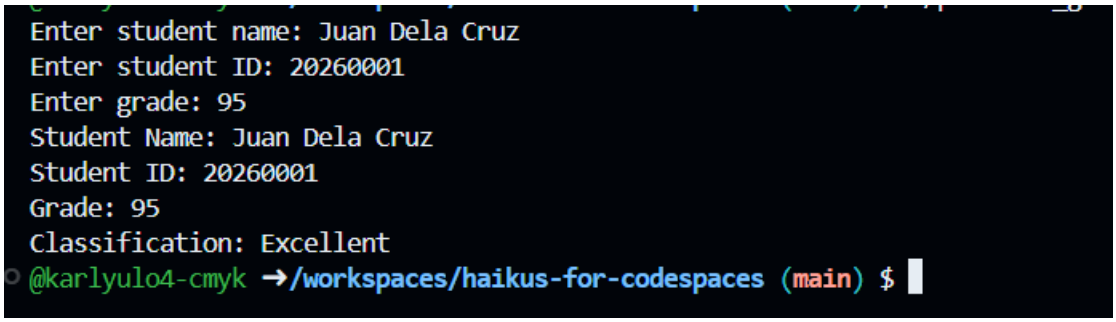
Grade range	Classification
90–100	Excellent
85–89	Very Good
80–84	Good
75–79	Passed
0–74	Failed
Outside 0–100 or nonnumeric	INVALID GRADE

Required test cases

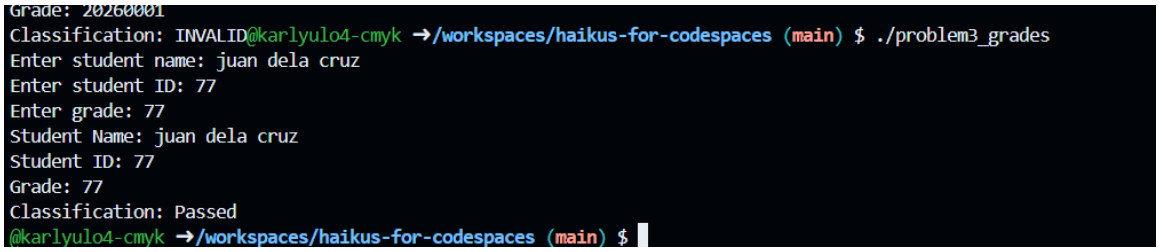
```
Enter student name: Juan Dela Cruz
Enter student ID: 20260001
Enter grade: 95
Student Name: Juan Dela Cruz
Student ID: 20260001
Grade: 95
Classification: Excellent
```

Also test grades 77 (Passed), 68 (Failed), 105 (INVALID GRADE), and 85 (Very Good).

Test 1:



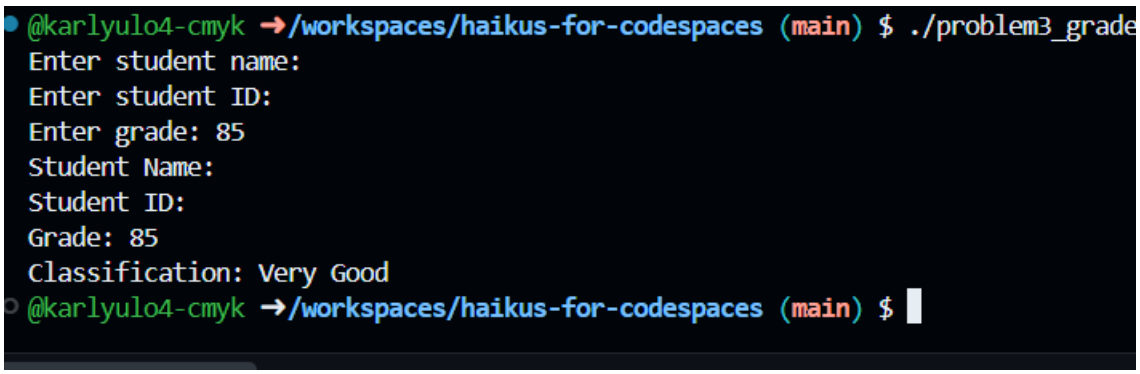
Test 2:



Test 3:



Test 4:



Student questions

• Why must ASCII digits be converted before comparing grade ranges?

ASCII digits must be converted into numeric values because the input from the keyboard is stored as characters. And converting them will allow the computer to read the inputs

• In what order did you check the grade boundaries, and why?

From highest to lowest so that the logic will track and wont do repeats

• How would you handle a grade of exactly 75 or 90?

When I searched there was a check option which will lead to it being checked if it also reaches that it will register in that range.

Problem 4 — ATM Withdrawal Validation

Difficulty: Intermediate–Challenging | Objective: Combine authentication, arithmetic, and nested validation.

Simulate an ATM with starting balance PHP 10000 and demonstration PIN 1234. Ask for account name and PIN. If PIN is wrong, display ACCESS DENIED and terminate. Otherwise show ACCESS GRANTED and the balance, then request a withdrawal amount. Check in this order: amount must be a positive integer; amount must not exceed balance; amount must be a multiple of 100. Show the corresponding error message or deduct the amount and print the new balance. This is a classroom simulation, not a secure banking implementation.

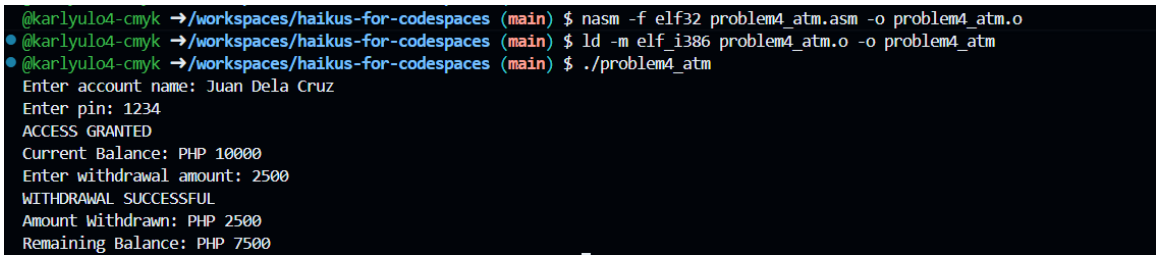
Condition	Output
Incorrect PIN	ACCESS DENIED
Amount ≤ 0 or nonnumeric	INVALID AMOUNT
Amount > balance	INSUFFICIENT FUNDS
Amount not divisible by 100	AMOUNT MUST BE A MULTIPLE OF 100
All checks pass	WITHDRAWAL SUCCESSFUL

Required test cases

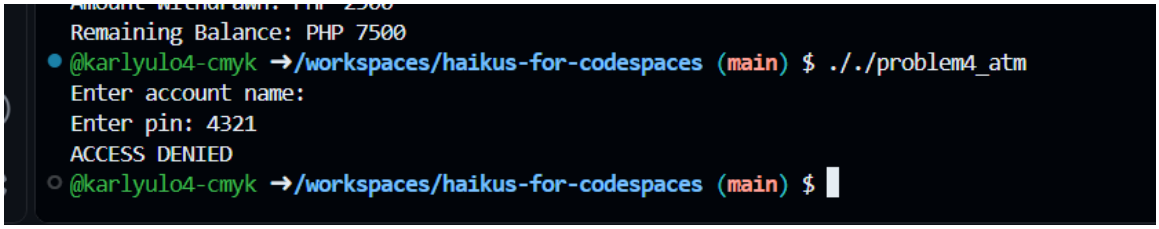
```
Enter account name: Juan Dela Cruz
Enter PIN: 1234
ACCESS GRANTED
Current Balance: PHP 10000
Enter withdrawal amount: 2500
WITHDRAWAL SUCCESSFUL
Amount Withdrawn: PHP 2500
Remaining Balance: PHP 7500
```

Also test PIN 4321 (ACCESS DENIED), withdrawal 15000 (INSUFFICIENT FUNDS), 2550 (AMOUNT MUST BE A MULTIPLE OF 100), and 0 (INVALID AMOUNT). Use PIN 1234 for withdrawal tests.

Test 1:



Test 2:



Test 3:



Test 4:



Test 5:



Student questions

• Why must PIN verification happen before displaying the account balance?

Because its to protect the withdrawal method because it is supposed to be with the right person and pin

• Which arithmetic instruction updates the balance after a successful withdrawal?

I used sub eax, [AMOUNT_val] subtracts the requested amount from the balance stored in EAX.

• How can a remainder be used to test whether an amount is a multiple of 100?

I used the div instruction and The remainder is stored in EDX and If EDX is zero, the amount is a multiple of 100 and if it is not it will trigger one of the methods of printing " AMOUNT MUST BE A MULTIPLE OF 100"

• What happens to the balance after a rejected withdrawal?

The balance remains the same

Submission Checklist

☐ Four .asm files, one per problem

☐ Compilation/linking evidence and test-case screenshots for every problem

☐ Answers to all analysis questions

☐ Programs use procedures, jumps, and appropriate input validation

Provide comprehensive Test Cases, include screenshot of each Test case

Suggested filenames: problem1_gender.asm, problem2_yearlevel.asm, problem3_grades.asm, problem4_atm.asm